Lecture 21: The Road to Attention

The fix that worked, and the wall it hit

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The disease, recalled

L20 ended on a number: influence fades as V^k, and $0.8^{29} \approx 0.0015$. Multiplying the same matrix into itself, 29 times, starved the gradient before it ever reached the first token.

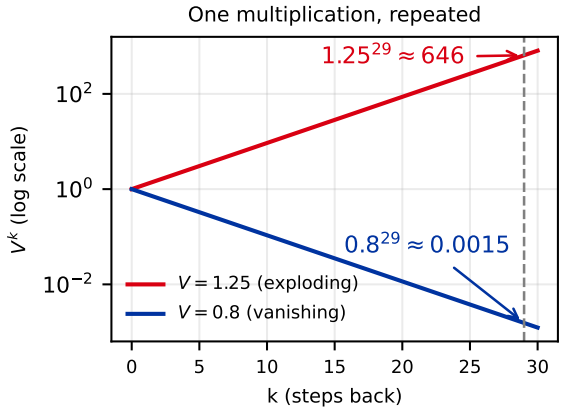
Predict: what arithmetic operation would *not* shrink a signal no matter how many times you repeated it?

The disease, recalled

L20 ended on a number: influence fades as V^k, and $0.8^{29} \approx 0.0015$. Multiplying the same matrix into itself, 29 times, starved the gradient before it ever reached the first token.

Predict: what arithmetic operation would *not* shrink a signal no matter how many times you repeated it?

Addition. Repeated multiplication decays (or explodes); repeated addition just accumulates. Hold that thought - it is the first half of this lecture, and you have already seen it once (ResNet's skip connection, [50]).



Multiplication: the disease. Addition: today's medicine.

Outline

LSTM: the additive fix

Language modeling

Seq2Seq and the wall

Epilogue: recurrence in 2026

A memory highway

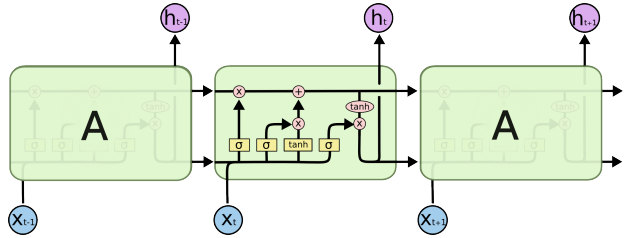
Hochreiter & Schmidhuber's 1997 answer to the disease Hochreiter's own 1991 thesis diagnosed.

The cell state

The **Long Short-Term Memory** (**LSTM**; Hochreiter & Schmidhuber, 1997) adds a second track running alongside the hidden state **z**: the **cell state c**. At every step it is only lightly modified - a little is removed, a little is added.

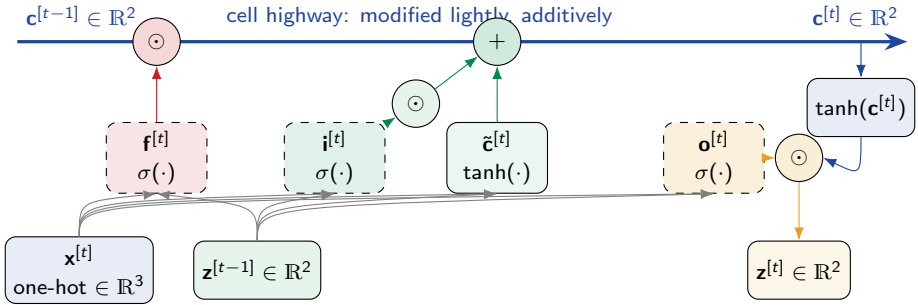
Sorting-line metaphor (the pomegranate line): the belt itself carries context past every station; each worker only adds or removes a little as the box goes by. Nobody rebuilds the box from scratch at every station.

Every horizontal line in the diagram is a full vector; circles are element-wise operations; boxes are learned layers.



Anatomy of one LSTM step

Same toy setup as L20's anatomy frame, plus a **2-d cell state**. Three gates and one candidate, each computed from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$; each gate is a sigmoid vector in $(0,1)^2$ used as an **element-wise mask** ($\odot$).

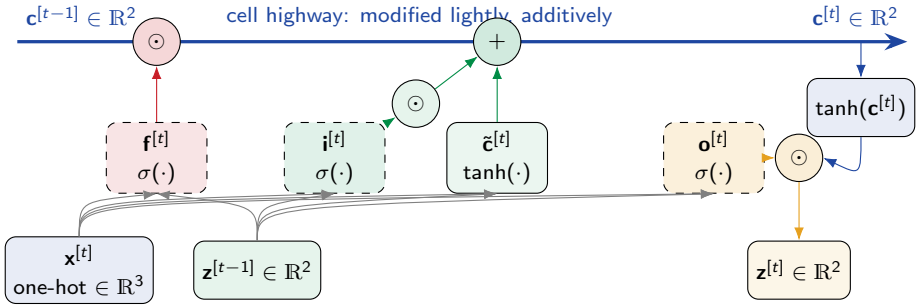


Forget: drop the old subject when a new one appears (LMU's gender-pronoun case). **Input:** write down a strong opinion word. **Output:** what the readout needs right now.

Predict: forget gate $[0.9, 0.1]$ meets cell state $[2.0, -1.0]$ - what survives?

Anatomy of one LSTM step

Same toy setup as L20's anatomy frame, plus a **2-d cell state**. Three gates and one candidate, each computed from $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$; each gate is a sigmoid vector in $(0,1)^2$ used as an **element-wise mask** ($\odot$).



Forget: drop the old subject when a new one appears (LMU's gender-pronoun case). **Input:** write down a strong opinion word. **Output:** what the readout needs right now.

Predict: forget gate $[0.9, 0.1]$ meets cell state $[2.0, -1.0]$ - what survives?

$[0.9, 0.1] \odot [2.0, -1.0] = [1.8, -0.1]$: memory 1 survives almost untouched, memory 2 is nearly wiped. A gate is [41_1]'s sigmoid neuron, rebranded as a soft switch.

For the record: the four equations

Reference, not exam material - shown once.

Forget gate: $\mathbf{f}^{[t]} = \sigma\big(\mathbf{W}_f^\top \mathbf{x}^{[t]} + \mathbf{V}_f^\top \mathbf{z}^{[t-1]} + \mathbf{b}_f\big)$

Input gate: $\mathbf{i}^{[t]} = \sigma\big(\mathbf{W}_i^\top \mathbf{x}^{[t]} + \mathbf{V}_i^\top \mathbf{z}^{[t-1]} + \mathbf{b}_i\big)$

$\tilde{\mathbf{c}}^{[t]} = \tanh\big(\mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{b}\big)$

Cell update: $\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \mathbf{i}^{[t]} \odot \tilde{\mathbf{c}}^{[t]}$

Output gate: $\mathbf{o}^{[t]} = \sigma\big(\mathbf{W}_o^\top \mathbf{x}^{[t]} + \mathbf{V}_o^\top \mathbf{z}^{[t-1]} + \mathbf{b}_o\big) \quad \mathbf{z}^{[t]} = \mathbf{o}^{[t]} \odot \tanh(\mathbf{c}^{[t]})$

The punchline: $\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \ldots$ is **additive** - gradients flow through the cell state with minimal decay. ResNet solved *depth* with addition in 2015; LSTM solved *time* with addition in 1997 - same medicine, 18 years earlier.

Why addition keeps the gradient alive

Follow the gradient backwards along the highway only (ignore the paths through the gates). One line of calculus:

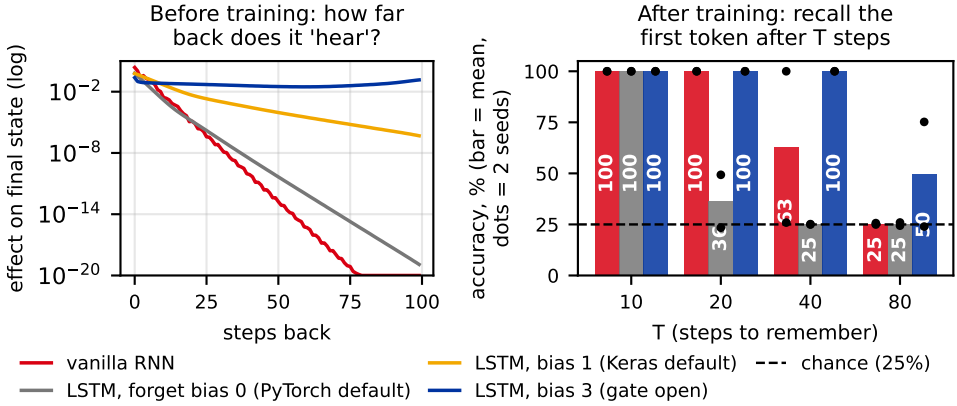
$$\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \mathbf{i}^{[t]} \odot \tilde{\mathbf{c}}^{[t]} \quad \Longrightarrow \quad \frac{\partial \mathbf{c}^{[t]}}{\partial \mathbf{c}^{[t-1]}} = \operatorname{diag}\left(\mathbf{f}^{[t]}\right)$$

Vanilla RNN (L20): each step multiplies by $\mathbf{D}^{[t-1]}\mathbf{V}^{\top}$ - the *same* matrix every step, whether or not the memory matters. Its size decides the fate of everything.

LSTM highway: each step multiplies by $\mathbf{f}^{[t]}$ - a gate the network *recomputes* from the input at every step, per memory slot. It can hold $f \approx 1$ while a memory matters (it survives 100 steps) and drop to $f \approx 0$ to wipe it on purpose.

History: the 1997 LSTM had *no* forget gate - its cell passed the gradient back with factor exactly 1 (a "constant error carousel"). The forget gate came in 2000 (Gers, Schmidhuber & Cummins), so the memory could also be cleared.

Does it work? Measured



Left, before any training: an input 40 steps back barely moves a vanilla RNN, nor an LSTM whose forget gates start half-shut; bias 1 still decays, only slower; open the gates (bias 3) and it still counts after 100 steps. **Right**, same budget for all: the open-gate LSTM solves $T \leq 40$ within 100 steps; the RNN is a coin flip at 40, at chance by 80. At 80 even the LSTM succeeds on one seed of two - addition makes long memory *learnable*, not free.

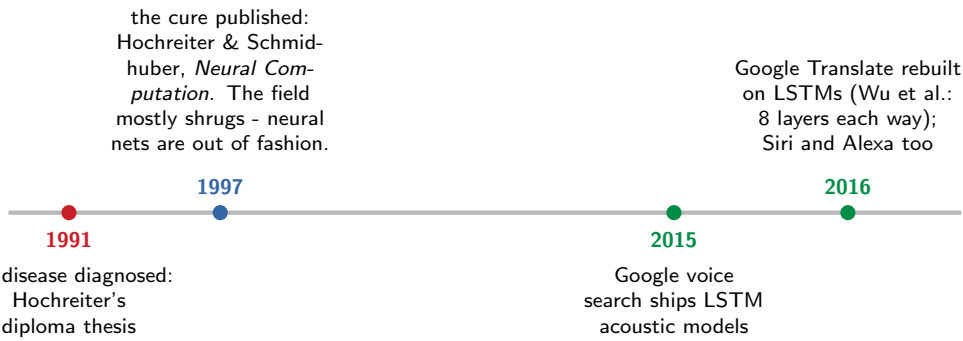
The price, and one gotcha

- **Price:** 4 weight sets instead of 1. Our d_x=1, d_h=16 vanilla cell: 16+256+16 = 288 params. LSTM: $4 \times 288 = \mathbf{1{,}152}$.
- **Gated Recurrent Unit** (**GRU**; Cho et al., 2014): a streamlined 2-gate cousin (3 weight sets), similar results - the lighter default.
- **Gotcha:** the left panel's "gate starts open" is a one-line choice. Keras adds +1 to the forget-gate bias by default; **PyTorch does not**:

```
lstm = nn.LSTM(d_in, hidden)
# PyTorch stacks the gates: input, forget, cell, output
lstm.bias_ih_l0.data[hidden:2 * hidden].fill_(1.0)  # forget gate
```

Same lesson as [45]'s initialization frames: where training *starts* can decide whether it gets anywhere at all.

Published 1997, everywhere by 2016



18 years from publication to running in everyone's pocket - and one of the most-cited AI papers ever (80,000+ citations). When a 2010s product said "powered by neural networks", the network was very often an LSTM.

Fine print, for next chapter: 2016's Google Translate already carried an *attention* link between its encoder and decoder - the idea beyond LSTM was creeping in even at LSTM's moment of triumph.

One task to rule them all

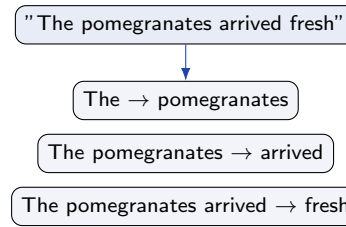
Predict the next token. That is the whole objective.

The objective

$$P(y_1, \ldots, y_T) = \prod_{i=1}^{T} P(y_i \mid y_1, \ldots, y_{i-1})$$

Turns raw text into supervised data **for free**: every prefix is an input, every next token a label. No labeling budget, no annotators - just more text.

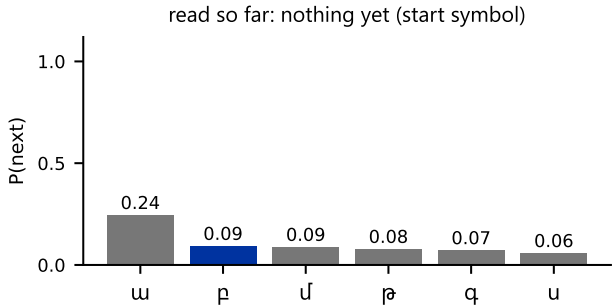
Said slowly, once: this exact objective, scaled up, is what GPT trains on.



every prefix → next-token pair, free supervision

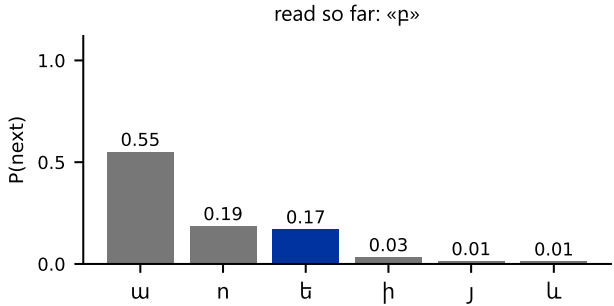
The mechanics, click by click

The ch11 name inventor's 689 Armenian surnames again, now read by a small **GRU** (the LSTM's lighter cousin from earlier) - trained, measured. At every step it outputs a probability for each possible next letter. A surname it **never saw in training**, one letter per click:



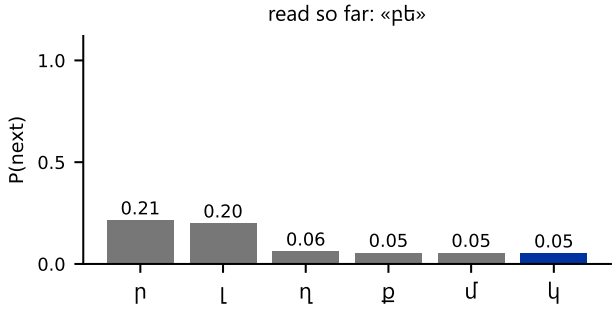
The mechanics, click by click

The ch11 name inventor's 689 Armenian surnames again, now read by a small **GRU** (the LSTM's lighter cousin from earlier) - trained, measured. At every step it outputs a probability for each possible next letter. A surname it **never saw in training**, one letter per click:



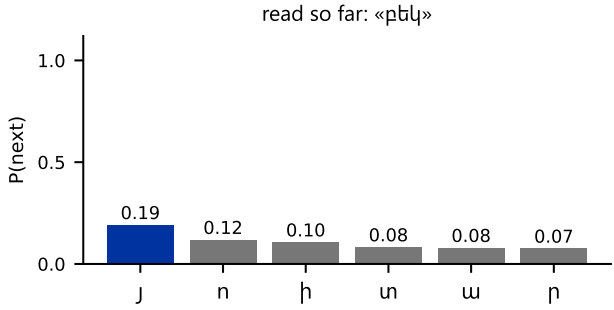
The mechanics, click by click

The ch11 name inventor's 689 Armenian surnames again, now read by a small **GRU** (the LSTM's lighter cousin from earlier) - trained, measured. At every step it outputs a probability for each possible next letter. A surname it **never saw in training**, one letter per click:



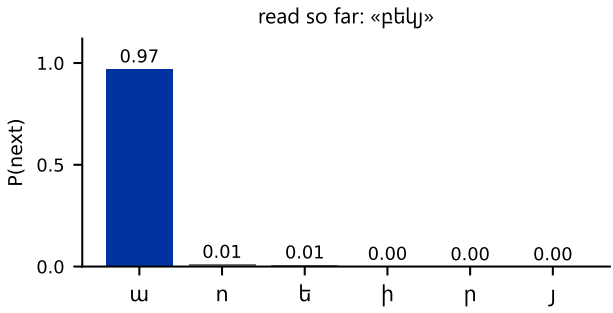
The mechanics, click by click

The ch11 name inventor's 689 Armenian surnames again, now read by a small **GRU** (the LSTM's lighter cousin from earlier) - trained, measured. At every step it outputs a probability for each possible next letter. A surname it **never saw in training**, one letter per click:



The mechanics, click by click

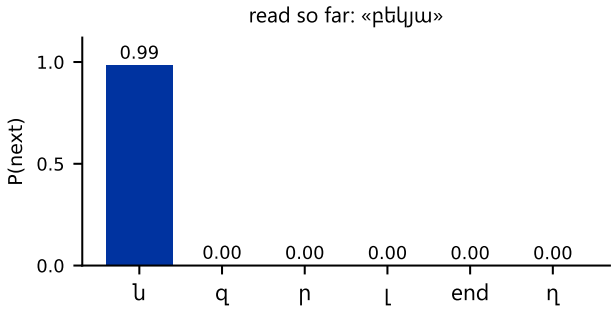
The ch11 name inventor's 689 Armenian surnames again, now read by a small **GRU** (the LSTM's lighter cousin from earlier) - trained, measured. At every step it outputs a probability for each possible next letter. A surname it **never saw in training**, one letter per click:



Unsure for four letters, near-certain once the ending starts: 95.8% of the surnames end in "-yan", and the network has learned it from the names alone.

The mechanics, click by click

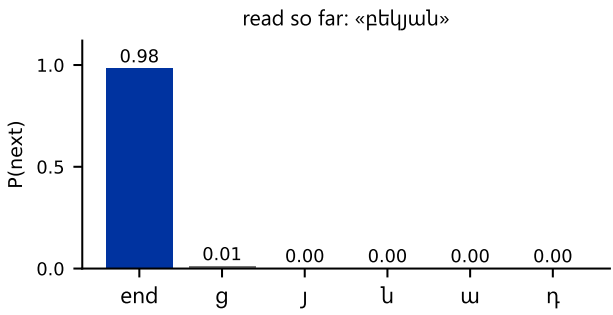
The ch11 name inventor's 689 Armenian surnames again, now read by a small **GRU** (the LSTM's lighter cousin from earlier) - trained, measured. At every step it outputs a probability for each possible next letter. A surname it **never saw in training**, one letter per click:



Unsure for four letters, near-certain once the ending starts: 95.8% of the surnames end in "-yan", and the network has learned it from the names alone.

The mechanics, click by click

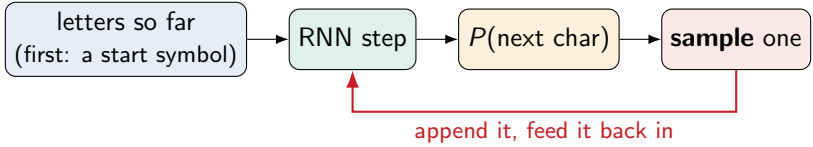
The ch11 name inventor's 689 Armenian surnames again, now read by a small **GRU** (the LSTM's lighter cousin from earlier) - trained, measured. At every step it outputs a probability for each possible next letter. A surname it **never saw in training**, one letter per click:



Unsure for four letters, near-certain once the ending starts: 95.8% of the surnames end in "-yan", and the network has learned it from the names alone.

From predicting to writing: sample, feed back, repeat

The network only ever **predicts** one distribution over the next character. *Writing* is a loop around it:



Repeat until the net samples the end symbol. Sample, rather than always take the most likely letter: the top choice every time writes the same text on every run. How to steer that choice is a topic of its own - the LLM chapter's decoding lecture.

This loop is exactly how ChatGPT writes: one token at a time, each one sampled and fed back in. Only the network inside the loop changes.

Demo: the GRU invents surnames

The same GRU and the loop above, sampled at five moments of its training - ten names each time, as they came out. **(Click to advance.)**

untrained (epoch 0) · val loss 3.65 nats/char

ժիգւբխքիէ… գխզգծգղւմ… ինբքչօգմև… յձիբչէւփօ…
զռփբչճիգէ… իգծկլթօհէւ… յնժեյսօրփ… ղօջձցշժռթ…
չշքեյզվիա… լսմչօաշյզ…

* = a real surname from the training set (0 of 10)

Demo: the GRU invents surnames

The same GRU and the loop above, sampled at five moments of its training - ten names each time, as they came out. **(Click to advance.)**

after epoch 1 · val loss 2.82 nats/char

պժվդփեզրր… ժտփրաչան ճպժջպգ նբգլրժթժշ…
ձվյձօնման… ժռհծամեծ ոյմվնտատս… օղմջւզտեգ…
ւլձո օգսև

* = a real surname from the training set (0 of 10)

Demo: the GRU invents surnames

The same GRU and the loop above, sampled at five moments of its training - ten names each time, as they came out. **(Click to advance.)**

after epoch 3 · val loss 2.29 nats/char

անդանոաբա… պմխն ւաթաւպճին… խվջրյ
ծաբսյւ նեղյյան աէփպկ նգիզեյ
կոկբասզրո… ետիանֆրան…

* = a real surname from the training set (0 of 10)

Demo: the GRU invents surnames

The same GRU and the loop above, sampled at five moments of its training - ten names each time, as they came out. **(Click to advance.)**

after epoch 10 · val loss 1.73 nats/char

ջթրոոյոյ… չրաշուևցչ… դարարյքն ապեյան
աիարքջյան աղավարյւնց նուրյան զվարյակյան
զադակոյան վանջազոնյ…

* = a real surname from the training set (0 of 10)

Demo: the GRU invents surnames

The same GRU and the loop above, sampled at five moments of its training - ten names each time, as they came out. **(Click to advance.)**

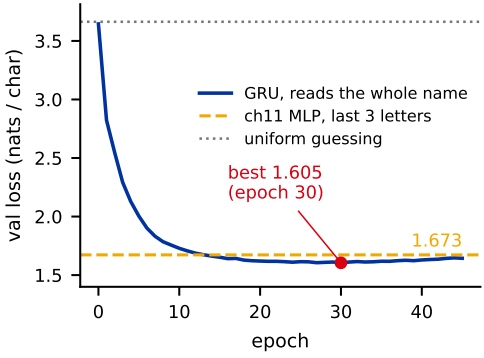
after epoch 30 - the best epoch · val loss 1.60 nats/char

ակոնի անամյրի քարուշյան տինաջյան
սուդարյան աղալետյան ասաբյան ալամուտյան
ալիզբյան մագրիսյան

* = a real surname from the training set (0 of 10)

None of the ten is copied from the training list: the network learned how Armenian surnames are *built*, not a list of them.

The loss curve, and a link



The next-letter objective, watched from the outside. Best: **1.605** nats per letter at epoch 30. The ch11 MLP, which saw only the last 3 letters: **1.673**.

Memory helps, but only a little: surnames are short, so 3 letters already see most of what matters. The practical measures where it pays.

The same objective on longer text: Karpathy (2015), **"The Unreasonable Effectiveness of Recurrent Neural Networks"** - char-RNNs writing Shakespeare and Linux source. `karpathy.github.io/2015/05/21/rnn-effectiveness/`

Two networks, one vector between them

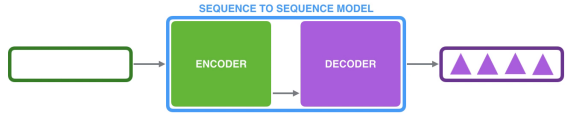
Translation needs a different-length output. Here is the classic fix.

Encoder-decoder

Translation needs different input/output lengths. Fix: an **encoder** LSTM reads the source and hands ONE **context vector** C to a **decoder** LSTM that writes the target.

The 2014 recipe that made neural machine translation real: Sutskever, Vinyals & Le (2014); Cho et al. (2014).

Running example: "I watch all of this in silence, and the connoisseurs speak inside me." goes in - and the decoder writes out the chapter's Armenian line from L20, word by word.

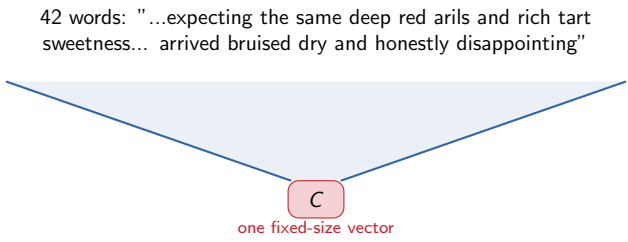


The bottleneck

Predict: the 42-word complaint review (L20's shoehorn-hacks frame - it lost its verdict to truncation already) squeezed into a single 300-dim context vector. What goes wrong?

The bottleneck

Predict: the 42-word complaint review (L20's shoehorn-hacks frame - it lost its verdict to truncation already) squeezed into a single 300-dim context vector. What goes wrong?

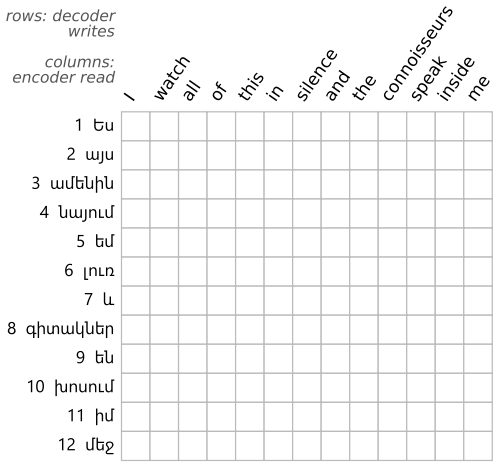


Lost: early words get overwritten as the state updates sequentially - "expecting", "childhood markets" fade first

Fixed capacity: 5 words or 500, the same-size C must hold them all

And the LSTM does NOT save us: the cell state is still one fixed-size vector. The fix for *fading* memory did not fix *compressed* memory.

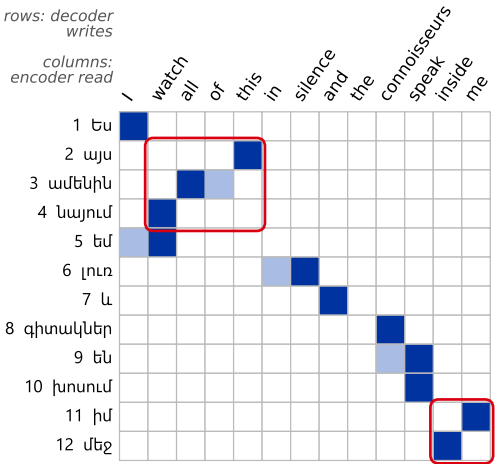
The real problem: every step gets the same C



So just make C bigger? First look at what the decoder actually *needs*.

Predict: the decoder writes the Armenian line one word per row. For each row, which English words does it need?

The real problem: every step gets the same C



So just make C bigger? First look at what the decoder actually *needs*.

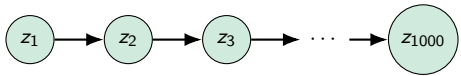
Predict: the decoder writes the Armenian line one word per row. For each row, which English words does it need?

Each row needs **one or two** words - a **different** few every time, and not even in order: rows 2-4 run backwards ("this all watch"), and so do rows 11-12 ("my inside").

Seq2seq hands **every** step the **same** C. The deeper problem is not that C is small: each step needs a **different part** of the input, and nothing lets the decoder go and get it.

The second wall: one step at a time

Even a perfect LSTM has one more problem, and it is about **speed**, not memory: $\mathbf{z}^{[t]}$ needs $\mathbf{z}^{[t-1]}$, which needs $\mathbf{z}^{[t-2]}$ ... Step t cannot start until step $t-1$ has finished.



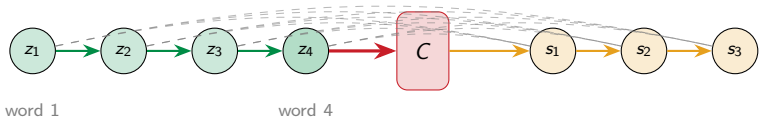
1,000 tokens = 1,000 steps
in a queue, one after another

- The GPUs from [42] want thousands of *independent* multiply-adds at once. A recurrence hands them one small step at a time - most of the chip waits.
- The authors of the paper that ended the RNN era put it plainly: "This inherently sequential nature precludes parallelization within training examples" (Vaswani et al., 2017).

Two walls, then: the **bottleneck** (one vector, the same for every step, must hold everything) and the **queue** (one step at a time). The next chapter's idea removes both at once.

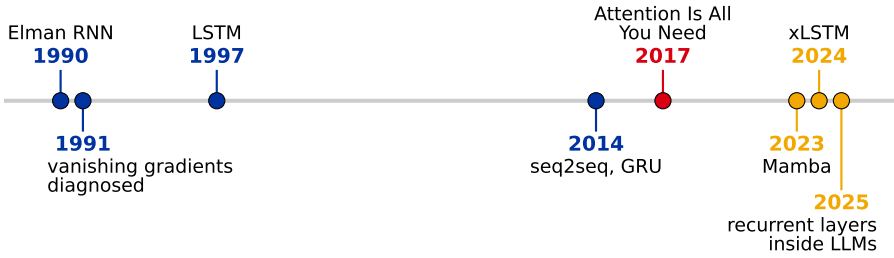
The cliffhanger

What if the decoder could look back at *all* the encoder states - and learn *where* to look, a different place for every word it writes?



That question has a name. It was answered between 2014 and 2017, and it ends this course's pre-history of sequence models. Next chapter: attention and transformers.

Recurrence, retirement, and a comeback



2017's transformer retired RNNs from mainstream NLP. But recurrence is back: several 2025-26 LLMs run most of their layers as a fixed-size recurrent state with a learned *decay* gate - today's forget gate. Why it came back makes sense only once you know what it replaced: the LLM chapter.

Recap

- Length varies, memory fades (L20) $\rightarrow$ LSTM fixes fading with an **additive** cell state - same medicine as ResNet, 18 years earlier - at $4\times$ the parameter cost.
- One objective - **predict the next token** - turns raw text into free training data; a small GRU learned how Armenian surnames are built from it, and at a different scale it trains GPT. **Writing** is the loop sample $\rightarrow$ feed back $\rightarrow$ repeat.
- **Seq2seq** squeezes the whole source sentence into one vector C and hands **every** output step that **same** C - while each step needs a **different** part of the input. And any recurrence runs **one step at a time**.

Next: the LLM chapter. It starts where text enters the model - what exactly is a token? - and then answers today's question: let the decoder look back at everything and take a different part for each word it writes. That is attention (LLM-3); dropping the recurrence altogether also removes the one-step-at-a-time queue.